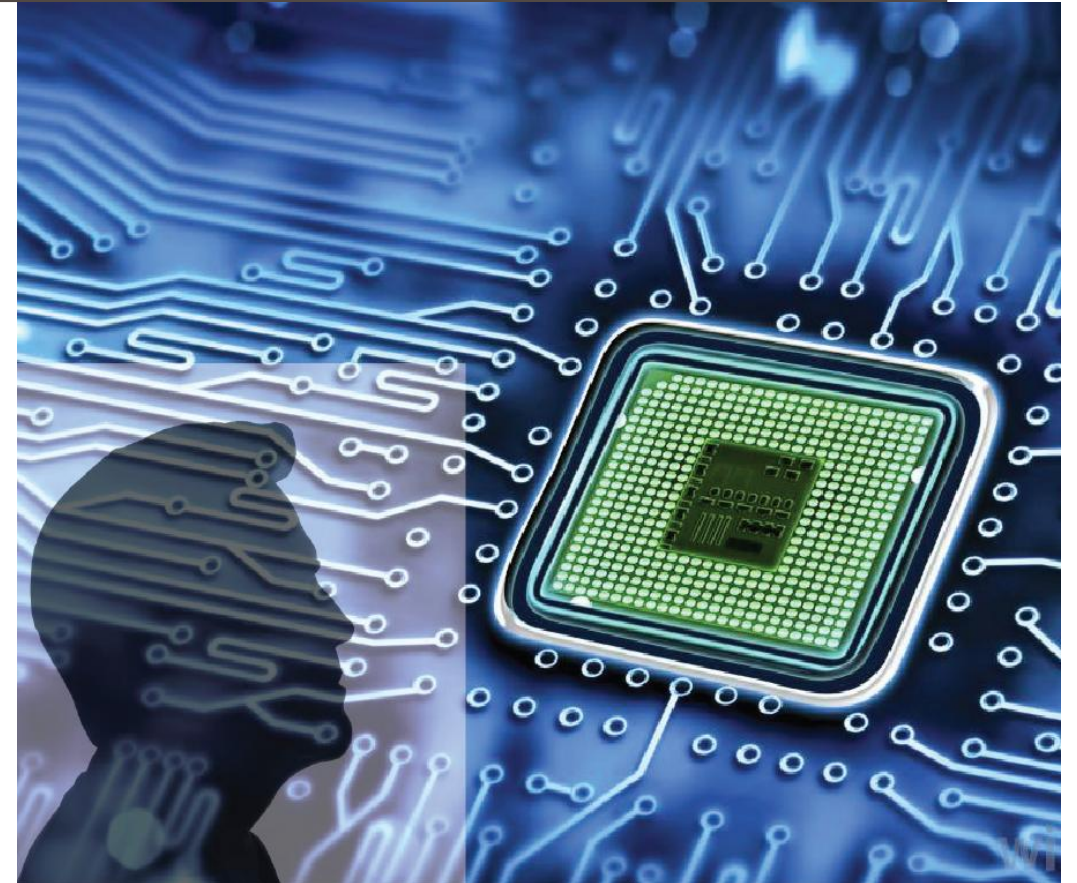


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-11 Lecture-2 SystemVerilog

Sneh Saurabh
6th November 2025



Clarification: Intersect (1)

```

class Generator;
  rand bit [1:0]x;
  rand bit [1:0]y;
  covergroup MyCover;
    coverpoint x {bins x[]={0:3}};
    coverpoint y {bins y[]={0:3}};
    c1: cross x, y {
      ignore_bins ib = binsof(x) intersect {[2:3]} && binsof(y);
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("x: %0d y: %0d", x, y);
  endfunction
endclass

```

```

ncsim> run
x: 1 y: 1
x: 0 y: 2
x: 0 y: 1
x: 2 y: 3
x: 2 y: 2
x: 3 y: 2
x: 0 y: 3
x: 3 y: 2
x: 1 y: 3
x: 3 y: 0
Coverage:

```

88

```

22 program MyRand();
23   initial begin
24     Generator gen;
25     gen = new(); // create object with random data
26     for (int i=0; i <10; i++) begin
27       gen.randomize();
28       gen.MyCover.sample();
29       gen.display();
30     end
31     $display("Coverage: %0d", gen.MyCover.get_coverage());
32   end
33 endprogram

```

x	y	Overall Average Grade
		(no filter)
x[0]	y[0]	! 0%
x[0]	y[1]	✓ 100%
x[0]	y[2]	✓ 100%
x[0]	y[3]	✓ 100%
x[1]	y[0]	! 0%
x[1]	y[1]	✓ 100%
x[1]	y[2]	! 0%
x[1]	y[3]	✓ 100%

Clarification: Intersect (2)

```

1 class Generator;
2   rand bit [1:0]x;
3   rand bit [1:0]y;
4   covergroup MyCover;
5     coverpoint x {bins x[]={0:3}};
6     coverpoint y {bins y[]={0:3}};
7     c1: cross x, y {
8       ignore_bins ib = binsof(x) intersect {[2:3]} || binsof(y) intersect {[0:2]};
9     }
10
11 endgroup
12
13 function new();
14   MyCover = new();
15 endfunction
16
17 function void display;
18   $display("x: %0d y: %0d", x, y);
19 endfunction
20 endclass

```

```

ncsim> run
x: 1 y: 1
x: 0 y: 2
x: 0 y: 1
x: 2 y: 3
x: 2 y: 2
x: 3 y: 2
x: 0 y: 3
x: 3 y: 2
x: 1 y: 3
x: 3 y: 0
Coverage: 100

```

x	y	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)
x[0]	y[3]	 100%	1 / 1 (10...	1
x[1]	y[3]	 100%	1 / 1 (10...	1

Running Coverage Tools from Synopsys: VCS

```
[sneh@edatools-server1 ~/system-verilog]$ vcs -sverilog intersect-ignore-1.sv -o simv
Chronologic VCS (TM)
Version Q-2020.03-SP2-4 -- Thu Nov  6 14:20:48 2025
Copyright (c) 1991-2020 by Synopsys Inc.
ALL RIGHTS RESERVED
```

This program is proprietary and confidential information of Synopsys Inc. and may be used and disclosed only as authorized in a license agreement controlling such use and disclosure.

```
Parsing design file 'intersect-ignore-1.sv'
Top Level Modules:
    MyRand
No TimeScale specified
Starting vcs inline pass...
```

```
1 module and 0 UDP read.
    However, due to incremental compilation, no re-compilation is necessary.
rm -f _cuarc*.so _csrc*.so pre_vcsobj_*.so share_vcsobj_*.so
ld -shared -z notext -m elf_i386 -Bsymbolic -o ../../simv.daidir//_cuarc0.so objs/amcQw_d.o
rm -f _cuarc0.so
if [ -x ../simv ]; then chmod a-x ../simv; fi
g++ -o ../simv -m32 -m32 -rdynamic -Wl,-rpath='$ORIGIN'/simv.daidir -Wl,-rpath=../simv.daidir
-Wl,-rpath=/usr/local/synopsys/vcs/vcs/Q-2020.03-SP2-4/linux/lib -L/usr/local/synopsys/vcs/vcs/Q-2
020.03-SP2-4/linux/lib -Wl,-rpath-link=../_40141_archive_1.so _prev_archive_1.so _cuarc0.so S
IM_l.o rmapats_mop.o rmapats.o rmar.o rmar_nd.o rmar_llvm_0_1.o rmar_llvm_0_0.o -lv
irsim -lerrorinf -lsnpsmalloc -lvfs -lvcsnew -lsimprofile -luclivative /usr/local/synopsys/vcs/v
cs/Q-2020.03-SP2-4/linux/lib/vcs_tls.o -Wl,-whole-archive -lvcsucli -Wl,-no-whole-archive
/usr/local/synopsys/vcs/vcs/Q-2020.03-SP2-4/linux/lib/vcs_save_restore_new.o /usr/local/synop
sys/vcs/vcs/Q-2020.03-SP2-4/linux/lib/ctype-stubs_32.a -ldl -lc -lm -lpthread -ldl
../simv up to date
CPU time: .182 seconds to compile + .190 seconds to elab + .259 seconds to link
```

Give filename intersect-ignore-1.sv appropriately

Running Coverage Tools from Synopsys: simulate

```
[sneh@edatools-server1 ~/system-verilog]$ ./simv
Chronologic VCS simulator copyright 1991-2020
Contains Synopsys proprietary information.
Compiler version Q-2020.03-SP2-4; Runtime version Q-2020.03-SP2-4; Nov 6 14:22 2025
x: 1 y: 0
x: 3 y: 0
x: 2 y: 2
x: 3 y: 2
x: 0 y: 0
x: 3 y: 2
x: 2 y: 0
x: 3 y: 2
x: 0 y: 3
x: 1 y: 3
Coverage: 75
$finish at simulation time 0
VCS Simulation Report
Time: 0
CPU Time: 0.370 seconds; Data structure size: 0.0Mb
```

simv was generated in the previous step.

```
class Generator;
  rand bit [1:0]x;
  rand bit [1:0]y;
  covergroup MyCover;
    coverpoint x {bins x[]={0:3}};
    coverpoint y {bins y[]={0:3}};
    cl: cross x, y {
      ignore_bins ib = binsof(x) intersect {[2:3]} && binsof(y);
    }
  endgroup

  function new();
    MyCover = new();
  endfunction

  function void display;
    $display("x: %0d y: %0d", x, y);
  endfunction
endclass
```

Running Coverage Tools from Synopsys: Verdi

```
[sneh@edatools-server1 ~/system-verilog]$ /usr/local/synopsys/2024/verdi/verdi/W-2024.09-SP2/bin/verdi -cov -covdir simv.vdb/ &
```

The screenshot displays the Synopsys Verdi coverage tool interface with three main panels:

- Summary Panel:** Shows overall coverage statistics and a table of coverage groups.
- CovSrc.1: \$u...tor::MyCover Panel:** Displays the Verilog source code with coverage annotations.
- CovDetail Panel:** Provides a detailed view of the coverage groups and their instances.

Summary Panel Data:

Group	Score	Instances	U+C
Cg \$unit::Generator::MyCover	75.00%	16	
Cp x	100.00%	4	
Cp y	75.00%	4	
Cr c1	50.00%	8	

CovSrc.1: \$u...tor::MyCover Panel Code:

```
4  covergroup
5  up MyCover;
6  coverpoint x {bins x
7  []={0:3}};
8  coverpoint y {bins y
9  []={0:3}};
10 c1: cross x, y {
11     ignore_b
12     ins ib = binsof(x)
13     intersect {[2:3]} &&
14     binsof(y);
15 }
16 endgroup
17 function
18 new();
19 MyCover = new();
```

CovDetail Panel Data:

Cover Group Item	Score	Instances	U+C	U
Cg \$unit::Genera...	75.00%		16	
Cp x	100.00%		4	
Cp y	75.00%		4	
Cr c1	50.00%		8	

CovDetail Panel Bin Data:

Status	Bin Name	x	y	Type	At
✓		x_0	y_0	Auto	
✓		x_0	y_3	Auto	
✓		x_1	y_0	Auto	
✓		x_1	y_3	Auto	
✗		[x_0, x_1]	[y_1, y_2]	Auto	
✗	ib			User	

SystemVerilog Assertion (SVA)

Assertions: Introduction

- **Assertion:** a statement about the behavior of a design which we expect always to be true

Purposes of Assertion:

1. Validate the correctness of a design
2. Flag input stimuli which are inconsistent with the requirements
3. Ensure functional coverage

- SystemVerilog provides rich set of constructs to specify assertions
 - SystemVerilog Assertion (SVA)

- Basic SVA constructs:

1. `assert`
2. `assume`
3. `cover`
4. `restrict`

Assertions: Example

```
check_grant: assert property
    @(posedge clk) !(grant[0] && grant[1])
    else $error("Two Grants simulataneously!");
```

Applications:

- **Simulation:** simulator will show an error when it detects an assertion to be false
 - Such stimulus should exist in the testbench
- **Formal Property Verification (FPV) tool:** assertions are treated as proof targets
 - Report that proof has been established
 - Report a violation (counter example) if exists
 - Report that it could not conclude anything (could not solve)

Example: Simulation of Good Design

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b01;
    else if (req[1])
      grant <= 2'b10;
    else
      grant <= 2'b00;
  end
  check_grant: assert property
    (@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
endmodule
```

Design:

- Arbiter sets one of the bits of grant to which it wants to give access (based on request)
- Property specifies that two grants are not made simultaneously

Test bench: provides various values to the request

```
interface arb_if(input bit clk);
  logic [1:0] grant, req;
  bit reset;

  clocking cb @(posedge clk); // Declare cb
  output req;
  input grant;
endclocking

modport TEST (clocking cb, // Use cb
              output reset);
endinterface

module top;
  bit clk;
  always #50 clk = ~clk;

  arb_if arbif(clk);

  arbiter a1 (.reset(arbif.reset), .clk(arbif.clk),
             .grant(arbif.grant), .req(arbif.req));
  test t1(arbif.TEST);
endmodule

program automatic test(arb_if.TEST arbif);
initial begin

  ##1 arbif.cb.req <= 3;
  ##1 arbif.cb.req <= 2;
  ##1 arbif.cb.req <= 1;
  #500 $finish;
end
endprogram : test
```

```
ncsim> run
Simulation complete via $finish(1) at time 750 NS + 3
./sva-1.sv:42    #500 $finish;
ncsim> exit
```

Example: Simulation of Bad Design

```
module arbiter(input logic [1:0] req,
  input bit clk, input bit reset,
  output logic [1:0] grant);
always @(posedge clk or negedge reset)
begin
  if (reset)
    grant <= 2'b00;
  else if (req[0])
    grant <= 2'b01;
  else if (req[1])
    grant <= 2'b11;
  else
    grant <= 2'b00;
end
check_grant: assert property
  (@(posedge clk) !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
endmodule
```

```
program automatic test(arb_if.TEST arbif);
initial begin

  ##1 arbif.cb.req <= 3;
  ##1 arbif.cb.req <= 2;
  ##1 arbif.cb.req <= 1;
  #500 $finish;
end
endprogram : test
```

```
ncsim> run
ncsim: *E,ASRTST (./sva-1-good.sv,16): (time 350 NS) Assertion top
a1.check_grant has failed
Two Grants simulataneously!
Simulation complete via $finish(1) at time 750 NS + 3
./sva-1-good.sv:40      #500 $finish;
```

Design:

- Arbiter sets two bits of grant in one of the cases

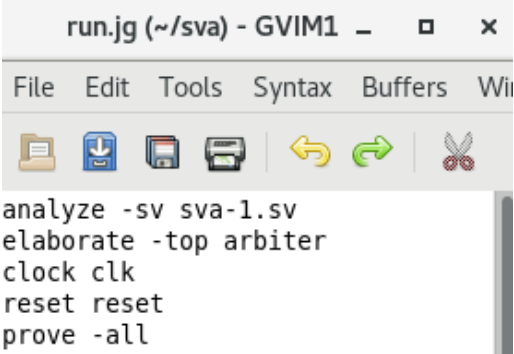
Example: Formal Property Verification

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b01;
    else if (req[1])
      grant <= 2'b10;
    else
      grant <= 2'b00;
  end
  check_grant: assert property
    (@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
endmodule
```

```
[sneh@edatools-server1 ~/sva]$ jaspergold -batch -tcl run.jg
JasperGold Apps 2019.09p002 64 bits 2019.11.26 18:17:27 PST
```

```
INFO (TPF057): 0.0.PRE: The property
was proven in 0.00 s.
```

Running Formal Property
Verification (FPV) tool
JasperGold



```
run.jg (~/sva) - GVIM1
File Edit Tools Syntax Buffers Win
analyze -sv sva-1.sv
elaborate -top arbiter
clock clk
reset reset
prove -all
```

=====

SUMMARY

=====

Properties Considered	: 1
assertions	: 1
- proven	: 1 (100%)
- bounded_proven (user)	: 0 (0%)
- bounded_proven (auto)	: 0 (0%)
- marked_proven	: 0 (0%)
- cex	: 0 (0%)
- ar_cex	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)
covers	: 0
- unreachable	: 0
- bounded_unreachable (user)	: 0
- covered	: 0
- ar_covered	: 0
- undetermined	: 0
- unknown	: 0
- error	: 0

proven

Example: Formal Property Verification (Bad Design)

```
module arbiter(input logic [1:0] req,
  input bit clk, input bit reset,
  output logic [1:0] grant);
always @(posedge clk or negedge reset)
begin
  if (reset)
    grant <= 2'b00;
  else if (req[0])
    grant <= 2'b01;
  else if (req[1])
    grant <= 2'b11;
  else
    grant <= 2'b00;
end
check_grant: assert property
  (@(posedge clk) !(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
endmodule
```

Design:

- Arbiter sets two bits of grant in one of the cases

```
0.0.N: Starting proof for property "arbiter.check_grant" [0.00 s].
0.0.Hp: clocks: 1, declared: 1, disabling: 0, posedge: 1, negedge: 0, noedge: 0, bothedge: 0
0.0.N: Trace Attempt 1 [0.00 s]
0.0.N: Requesting engine job to stop
0.0.Hp: Requesting engine job to terminate
INFO (IPF144): 0: Initiating shutdown of proof [0.00]
0.0.N: A trace with 1 cycles was found. [0.00 s]
INFO (IPF055): 0.0.N: A counterexample (cex) with 1 cycles was found for the property "arbiter.check_grant" in 0.00 s.
```

SUMMARY

```
Properties Considered : 1
assertions : 1
- proven : 0 (0%)
- bounded_proven (user) : 0 (0%)
- bounded_proven (auto) : 0 (0%)
- marked proven : 0 (0%)
- cex : 1 (100%)
- ar_cex : 0 (0%)
- undetermined : 0 (0%)
- unknown : 0 (0%)
- error : 0 (0%)
covers : 0
- unreachable : 0
- bounded_unreachable (user): 0
- covered : 0
- ar_covered : 0
- undetermined : 0
- unknown : 0
- error : 0
```

cex

Assumption

- Assumption: a statement that specify conditions that arise due to constraints on the environment
- The syntax for assumption and assertion is the same (SVA)

```
assume_second_requestor_off: assume property(@(posedge clk) (req[1] == 1'b0)  
    else $error("Second requestor ON");
```

Assumption: Simulation

- Simulator treats assumption similar to assertion and will show an error when it detects an assumption to be false
- Failed assumption typically indicate problem in the testing environment/code or neighboring module that provides input

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b01;
    else if (req[1])
      grant <= 2'b11;
    else
      grant <= 2'b00;
  end
  check_grant: assert property(@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
  assume_second_request: assume property(@(posedge clk) (req[1] == 1'b0))
    else $error("Second requestor ON");
endmodule
```

```
program automatic test(arb_if.TEST arbif);
initial begin
  ##1 arbif.cb.req <= 3;
  ##1 arbif.cb.req <= 2;
  ##1 arbif.cb.req <= 1;
  #500 $finish;
end
endprogram : test
```

```
ncsim> run
ncsim: *E,ASRTST (./sva-assume-1.sv,18): (time 50 NS) Assertion top.a1.assume_s
econd_request has failed
Second requestor ON
ncsim: *E,ASRTST (./sva-assume-1.sv,16): (time 350 NS) Assertion top.a1.check_g
rant has failed
Two Grants simulataneously!
Simulation complete via $finish(1) at time 750 NS + 3
```


Assumption: FPV

- Formal Property Verification (FPV) tool: assumptions are assumed to be true
 - Assumptions are treated as axioms and assertions are proven using the assertions
 - Eliminates unrealistic behavior
 - Writing assumption can be useful for a designer to guide the FPV tool to look in the interesting regions

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b01;
    else if (req[1])
      grant <= 2'b11;
    else
      grant <= 2'b00;
  end
  check_grant: assert property(@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simultaneously!");
  assume_second_request: assume property(@(posedge clk) (req[1] == 1'b0))
    else $error("Second requestor ON");
endmodule
```

0.0.PRE: A proof was found: No trace exists. [0.00 s]
INFO (IPF057): 0.0.PRE: The property "arbiter.check_grant" was proven in 0.00 s.

=====

SUMMARY

=====

Properties Considered	: 1
assertions	: 1
- proven	: 1 (100%)
- bounded_proven (user)	: 0 (0%)
- bounded_proven (auto)	: 0 (0%)
- marked_proven	: 0 (0%)
- cex	: 0 (0%)
- ar_cex	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)
covers	: 0
- unreachable	: 0
- bounded_unreachable (user)	: 0
- covered	: 0
- ar_covered	: 0
- undetermined	: 0
- unknown	: 0
- error	: 0

proven

Cover Point

- Cover point: a statement that specify some interesting condition that we want to make sure that we are testing
- The syntax for cover is similar to assertion and assumption (SVA)

```
cover_simultaneous_request: cover  
    property(@(posedge clk) (req[0] && req[1]));
```

Cover Point: Simulation (1)

- Simulator gathers information about hitting a cover point during simulation
 - Can be examined using coverage tool such as IMC
 - We can determine cover points that are not hit and fix simulation holes

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
always @(posedge clk or negedge reset)
begin
    if (reset)
        grant <= 2'b00;
    else if (req[0])
        grant <= 2'b01;
    else if (req[1])
        grant <= 2'b11;
    else
        grant <= 2'b00;
end
check_grant: assert
    property(@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
assume_second_request: assume
    property(@(posedge clk) (req[1] == 1'b0))
    else $error("Second requestor ON");
cover_simultaneous_request: cover
    property(@(posedge clk) (req[0] && req[1]));
endmodule
```

```
program automatic test(arb_if.TEST arbif);
initial begin
    ##1 arbif.cb.req <= 0;
    ##1 arbif.cb.req <= 1;
    ##1 arbif.cb.req <= 3;
    #500 $finish;
end
endprogram : test
```

```
[sneh@edatools-server1 ~/sva]$ ncverilog sva-cover-1.sv -coverage all -covoverw
rite
```

```
ncsim> run
ncsim: *E,ASRTST (./sva-cover-1.sv,20): (time 50 NS) Assertion top.a1.assume_se
cond_request has failed
Second requestor ON
ncsim: *E,ASRTST (./sva-cover-1.sv,20): (time 350 NS) Assertion top.a1.assume_s
econd_request has failed
Second requestor ON
Simulation complete via $finish(1) at time 750 NS + 3
```

Cover Point: Simulation (2)

```
cover_simultaneous_request: cover
    property(@(posedge clk) (req[0] && req[1]));
```

```
program automatic test(arb_if.TEST arbif);
initial begin
    ##1 arbif.cb.req <= 0;
    ##1 arbif.cb.req <= 1;
    ##1 arbif.cb.req <= 3;
    #500 $finish;
end
endprogram : test
```

Assertions				
Ex	UNR	Name	Overall Average Grade	Assertion Status Grade
		(no filter)	(no filter)	(no filter)
		▶ cover_simultaneous_request	✓ 100%	100%
		▶ assume_second_request	✓ 100%	0%
		▶ check_grant	✓ 100%	100%

```
program automatic test(arb_if.TEST arbif);
initial begin
    ##1 arbif.cb.req <= 0;
    ##1 arbif.cb.req <= 1;
    #500 $finish;
end
endprogram : test
```

Assertions				
Ex	UNR	Name	Overall Average Grade	Assertion Status Grade
		(no filter)	(no filter)	(no filter)
		cover_simultaneous_request	! 0%	0%
		assume_second_request	100%	0%
		check_grant	100%	100%

Cover Point: FPV Tool

- Formal Property Verification (FPV) tool checks whether the environment is able to reach cover point
 - Assumptions can sometimes over-constrain the solution space in an unexpected manner
 - Cover point helps to detect over constrained environment

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
  always @(posedge clk or negedge reset)
  begin
    if (reset)
      grant <= 2'b00;
    else if (req[0])
      grant <= 2'b01;
    else if (req[1])
      grant <= 2'b11;
    else
      grant <= 2'b00;
  end
  check_grant: assert
    property(@(posedge clk) !(grant[0] && grant[1]))
    else $error("Two Grants simulataneously!");
  assume_second_request: assume
    property(@(posedge clk) (req[1] == 1'b0))
    else $error("Second requestor ON");
  cover_simultaneous_request: cover
    property(@(posedge clk) (req[0] && req[1]));
endmodule
```

0.0.PRE: A proof was found: No trace exists. [0.00 s]
INFO (IPF051): 0.0.PRE: The cover property "arbiter.cover simultaneous request"
was proven unreachable in 0.00 s.

SUMMARY

Properties Considered	: 2
assertions	: 1
- proven	: 1 (100%)
- bounded_proven (user)	: 0 (0%)
- bounded_proven (auto)	: 0 (0%)
- marked_proven	: 0 (0%)
- cex	: 0 (0%)
- ar_cex	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)
covers	: 1
- unreachable	: 1 (100%)
- bounded_unreachable (user)	: 0 (0%)
- covered	: 0 (0%)
- ar_covered	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)

determined

Assertion Construct Hierarchy (1)

Assertion Statements

Properties

Sequences

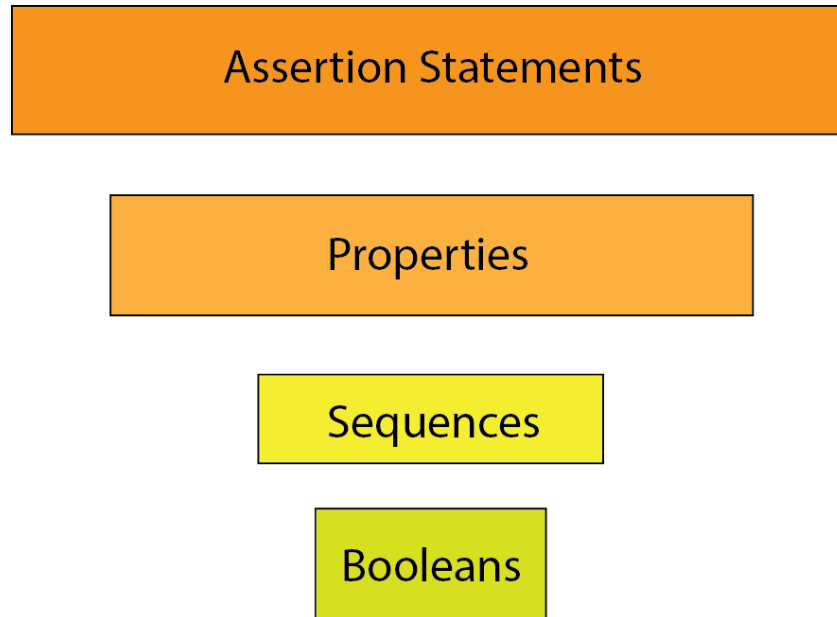
Booleans

- **Boolean:** Boolean variable or formula
 - Example: (req[0] && grant[0])

Table 3.1 Useful Boolean building blocks in the SystemVerilog language.

Construct	Meaning	Usage example
<code>\$countbits(expression, val1, val2, ...)</code> ^a	Count occurrences of <i>val</i> ^a	<code>num_gnts = \$countbits(gnt, 1'b1)</code>
<code>\$countones(expression)</code>	Count bits that are equal to 1	<code>num_gnts = \$countones(gnt)</code>
<code>\$onehot(expression)</code>	True iff precisely one bit is 1	<code>safe_gnts = \$onehot(gnt)</code>
<code>\$onehot0(expression)</code>	True iff no bits are 1, or precisely one bit is 1	<code>safe_or_no_gnts = \$onehot0(gnt)</code>
<code>expression inside list</code>	True iff expression is in list	<code>safe_opcode = opcode inside {ADD, SUB, MUL}</code>

Assertion Construct Hierarchy (2)



- **Sequences:** statements about Boolean values (or sequences) happening over time
 - Example: request followed by grant in two clock cycles
 - `req[0] ##2 grant[0]`

- **Properties:** specify some behavior to hold in a design (combine sequences with operators to show implication and similar concept)
 - Example: if request is followed by grant in two clock cycles, it implies that the grant will de-assert at the next clock cycle
 - `req[0] ##2 grant[0] |-> ##1 !grant[0]`

Assertion Construct Hierarchy (3)

Assertion Statements

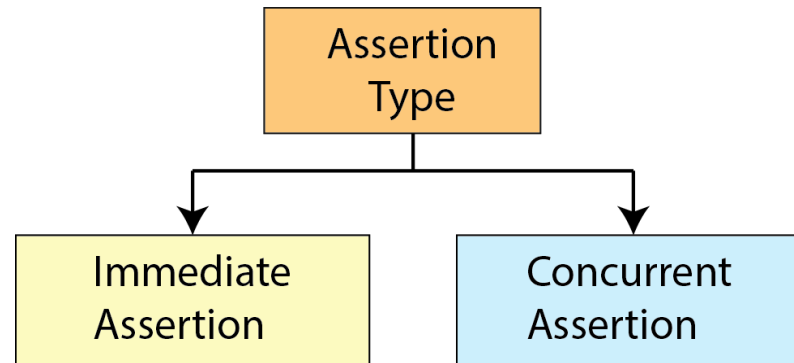
Properties

Sequences

Booleans

- Assertion: statement using one of the following keywords: **assert**, **assume**, or **cover**
 - `gnt_falls: assert property req[0] ##2 grant[0] |-> ##1 !grant[0]`

Types of Assertions



Immediate assertions:

- Simple assertions checked whenever they are visited in the procedural code
- Allow only Boolean arguments
- No mechanism of clocking or reset
- Do not support advance property operators (passage of time)
- Keyword **assert** without keyword **property**

Concurrent assertions:

- Always evaluated relative to one or more clocks
- Can describe behavior that spans time
 - Advanced operators such as logical implication over time intervals
- More useful than immediate assertion
- Keyword **assert** with keyword **property**

Immediate Assertion

- Immediate assertions are considered part of procedural code
 - Evaluated when encountered
- Does not have concept of clock
 - Can model simple situations

```
check_grant_immediate: assert final (!(grant[0] && grant[1]))  
                        else $error("Two Grants simulataneously!");
```

Interpretation

- Interpreted the same way as an expression in the condition of a procedural if statement.
 - If the expression evaluates to x, z, or 0, then it is considered to fail
 - Otherwise, the expression is interpreted as pass

An immediate assertion statement may be

- immediate **assert**
- immediate **assume**
- immediate **cover**

Applications

- Functions (example checking validity of arguments)
- Un-clocked logic

Immediate Assertion: Simulation Example

```
module arbiter(input logic [1:0] req,
               input bit clk, input bit reset,
               output logic [1:0] grant);
always @(*)
begin
    if (reset)
        grant = 2'b00;
    else if (req[0])
        grant = 2'b01;
    else if (req[1])
        grant = 2'b10;
    else
        grant = 2'b00;

    $display("At %0t reset: %b req: %0d grant: %0d expr: %0b",
            $time, reset, req, grant, (!(grant[0] && grant[1])));

    check_grant_immediate: assert final (!(grant[0] && grant[1]))
        else $error("Two Grants simulataneously!");
end
endmodule
```

```
if (reset)
    grant = 2'b00;
else if (req[0])
    grant = 2'b01;
else if (req[1])
    grant = 2'b11;
else
    grant = 2'b00;
```

```
program automatic test(arb_if.TEST arbif);
initial begin
    ##1 arbif.cb.req <= 3;
    ##1 arbif.cb.req <= 2;
    ##1 arbif.cb.req <= 1;
    #500 $finish;
end
endprogram : test
```

```
ncsim> run
At 0 reset: 0 req: x grant: 0 expr: 1
At 50 reset: 0 req: 3 grant: 1 expr: 1
At 150 reset: 0 req: 2 grant: 2 expr: 1
At 250 reset: 0 req: 1 grant: 1 expr: 1
```

```
ncsim> run
At 0 reset: 0 req: x grant: 0 expr: 1
At 50 reset: 0 req: 3 grant: 1 expr: 1
At 150 reset: 0 req: 2 grant: 3 expr: 0
ncsim: *E,ASRTST (./sva-immediate-1.sv,18): (time 150 NS) Assertion top.al.check_grant_immediate has failed
Two Grants simulataneously!
```

Immediate Assertion: FPV Example

```
if (reset)
  grant = 2'b00
else if (req[0])
  grant = 2'b01;
else if (req[1])
  grant = 2'b10;
else
  grant = 2'b00;
```

```
if (reset)
  grant = 2'b00;
else if (req[0])
  grant = 2'b01;
else if (req[1])
  grant = 2'b11;
else
  grant = 2'b00;
```

```
check_grant_immediate: assert final (!(grant[0] && grant[1]))
  else $error("Two Grants simulataneously!");
```

SUMMARY

Properties Considered	: 1
assertions	: 1
- proven	: 1 (100%)
- bounded_proven (user)	: 0 (0%)
- bounded_proven (auto)	: 0 (0%)
- marked_proven	: 0 (0%)
- cex	: 0 (0%)
- ar_cex	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)
covers	: 0
- unreachable	: 0
- bounded_unreachable (user)	: 0
- covered	: 0
- ar_covered	: 0
- undetermined	: 0
- unknown	: 0
- error	: 0

proven

SUMMARY

Properties Considered	: 1
assertions	: 1
- proven	: 0 (0%)
- bounded_proven (user)	: 0 (0%)
- bounded_proven (auto)	: 0 (0%)
- marked_proven	: 0 (0%)
- cex	: 1 (100%)
- ar_cex	: 0 (0%)
- undetermined	: 0 (0%)
- unknown	: 0 (0%)
- error	: 0 (0%)
covers	: 0
- unreachable	: 0
- bounded_unreachable (user)	: 0
- covered	: 0
- ar_covered	: 0
- undetermined	: 0
- unknown	: 0
- error	: 0

cex

0.0.PRE: A proof was found: No trace exists. [0.00 s]

INFO (IPF057): 0.0.PRE: The property "arbiter.check_grant_immediate" was proven in 0.00 s.

0.0.N: A trace with 1 cycles was found. [0.00 s]

INFO (IPF055): 0.0.N: A counterexample (cex) with 1 cycles was found for the property "arbiter.check grant immediate" in 0.01 s.

References

- Seligman, Erik, Tom Schubert, and MV Achutha Kiran Kumar. Formal verification: an essential toolkit for modern VLSI design. Elsevier, 2023
- Saurabh, Sneh. Introduction to VLSI design flow. Cambridge University Press, 2023.